

Revision — 1.1.1 Structure and Function of the Processor

OCR H446 — one-page revision summary

Must know

- The **ALU** performs arithmetic (+ − × ÷), logic (AND/OR/NOT, comparisons) and shifts; results go to the **ACC**.
- The **Control Unit (CU)** decodes instructions, generates control signals, manages data flow on the buses and synchronises the CPU with the clock.
- Key **registers**: PC (address of next instruction), MAR (address to read/write), MDR (data/instruction fetched or to be written), CIR (current instruction being decoded/executed), ACC (immediate ALU result). **MAR = address, MDR = data**.
- Three **buses**: **address** (unidirectional, CPU → memory), **data** (bidirectional), **control** (bidirectional). Address bus width → 2ⁿ addressable locations.
- **Clock speed** (Hz) = pulses/second; higher = more FDE cycles/second, but is limited by heat and power consumption.
- **Cores**: each is an independent unit running its own FDE cycle. More cores speed things up **only if the task is parallelisable and the software supports it**.
- **Cache** = small, very fast memory on/near the CPU holding frequently/recently used data — it is **not** part of RAM. Checked in order **L1 → L2 → L3 → RAM** (L1 smallest/fastest, L3 largest/slowest but still faster than RAM).
- **Pipelining** overlaps the fetch, decode and execute stages of consecutive instructions to raise throughput; **jump/branch** instructions cause hazards (prefetched instructions must be flushed).

Key terms

Term	Definition
Register	Small, very fast store inside the CPU holding data during processing
Program Counter (PC)	Holds the address of the next instruction; incremented each cycle
MAR	Memory Address Register — address of the location to be read/written
MDR	Memory Data Register — data/instruction just fetched or about to be written
Accumulator (ACC)	Stores the immediate result of ALU calculations
Cache	Small fast memory on/near the CPU holding frequently used data/instructions
Pipelining	Overlapping fetch/decode/execute of consecutive instructions for throughput
Core	Independent processing unit able to run its own FDE cycle

Worked example / how to — the Fetch–Decode–Execute cycle

FETCH 1. Contents of the **PC** are copied to the **MAR**. 2. A **read** signal is sent on the **control bus**; the instruction returns via the **data bus** into the **MDR**. 3. The **PC is incremented** (PC := PC + 1). 4. The

instruction is copied **MDR** → **CIR**.

DECODE — the CU splits the CIR contents into **opcode** (the operation) and **operand** (the data/address).

EXECUTE — the instruction is carried out: an ALU operation (result → ACC), a memory load/store, or a **jump** (a new address is written into the PC). The cycle then repeats.

Exam tips

- Never swap **MAR (address)** and **MDR (data)** — a very common lost mark.
- In a fetch trace, remember **both** the PC increment **and** the MDR → CIR copy — candidates routinely miss these.
- State the **address bus is unidirectional**; data and control buses are bidirectional.
- For "more cores = faster", always add the caveat: **only if the task parallelises and the software supports it**; describe cache as separate fast memory **on/near the CPU, not part of RAM**.